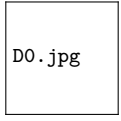
Lecture 17: The Physical Database

From Hardware & Storage to Query Optimization

Z2004: Database Management Systems



IIT Madras Zanzibar

Thursday, 9 April 2026

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain the **memory hierarchy** (RAM vs. SSD/HDD) and why **Disk I/O** dominates DBMS performance.
- Describe how a DBMS stores tables as **files of pages** (blocks), and why systems move pages (not rows) between disk and RAM.
- Sketch the **slotted page** layout (header, slot array, tuples, free space) and how it supports record lookup/movement.
- Explain what the **buffer pool manager** does (fetch, pin/unpin, evict, and write-back dirty pages).
- Compare a **full table scan** vs. an **index lookup** and reason about when scanning can be faster (low selectivity).
- Explain at a high level how **B+ trees** enable fast lookups with only a few page reads.
- Trace the **query processing pipeline**: SQL $\rightarrow$ Parser $\rightarrow$ Optimizer (cost model) $\rightarrow$ Execution Engine.

1. Introduction: Why Hardware Matters to SQL

The Real-World Problem

So far, we have treated databases like magic math boxes. You write a SQL query, and data instantly appears. But in the real world, data lives on physical metal, and **metal has limits.**

Example: Netflix Streaming

When you open Netflix, it must instantly load your specific "Continue Watching" list out of 250 million users.

- How does it find your record in milliseconds?
- Where is your viewing history actually stored when the servers lose power?

Example: Paying for a Flight

You click "Buy Ticket" on an airline app.

- The system must lock the seat, deduct your money, and update the database.
- If the server hardware crashes exactly when you click "pay," how does the DBMS guarantee your ticket isn't lost?

2. The Memory Hierarchy: RAM vs. Disk

Where does data live?

A DBMS operates across two entirely different types of hardware. Understanding the difference between them is the key to database performance.

1. Primary Memory (RAM)

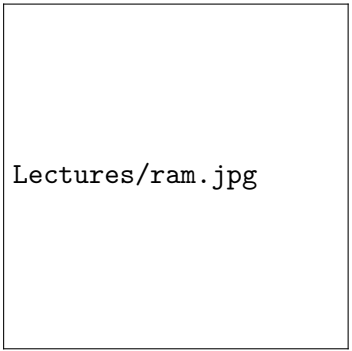
- **What it is:** The computer's "short-term memory" workspace.
- **Speed:** **Ultra-Fast** (Nanoseconds).
- **Capacity:** Small and Expensive ($\sim$16GB to 256GB).
- **Volatile:** If power goes out, **EVERYTHING IS LOST.**

2. Secondary Storage (Disk)

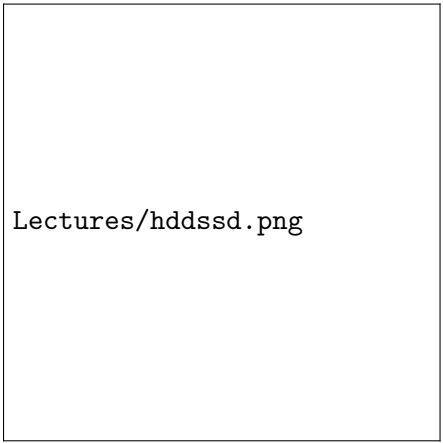
- **What it is:** The permanent filing cabinet (HDD/SSD).
- **Speed:** **Extremely Slow** (Milliseconds).
- **Capacity:** Massive and Cheap (Terabytes).
- **Non-Volatile:** Data survives power failures (Safe).

The CPU **cannot** process data directly on the Disk! Data MUST be moved from the Disk into the RAM before the database can read it, filter it, or update it.

RAM vs. HDD



RAM



HDD vs SSD

Latency - Why DBMS cares about hardware

Think in **orders of magnitude**:

Units: ns = nanosecond, μs = microsecond, ms = millisecond.
1000 ns = 1 μs (microsecond) 1,000,000 ns = 1 ms (millisecond)

Where?	Typical latency	DBMS implication
CPU register / L1 cache	$\sim$1 ns	practically free
RAM (buffer pool)	$\sim$100 ns	fast, but limited
SSD random read	$\sim$100 μs–1 ms	**expensive**
HDD random read (seek)	$\sim$5–10 ms	**very expensive**

Rule of thumb: one random disk read can cost **millions** of CPU cycles.

3. The Evolution of Disk Storage

How did we get from HDD to SSD?

For decades, databases were limited by the physical mechanics of **Hard Disk Drives (HDDs)**.

The HDD (Mechanical Era)

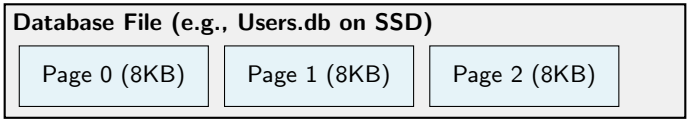
- **How it works:** HDDs have physical metal platters that spin at 7200 RPM, and a mechanical "arm" that reads magnets.
- **The Bottleneck:** If you want to read User #1's record, and then User #10,000's record, the physical arm has to literally move across the disk.
- This mechanical movement is called **Seek Time**. It is devastatingly slow for databases doing random lookups.

The SSD (Electrical Era)

- **How it works:** Solid State Drives use **NAND Flash memory** microchips. There are zero moving parts.
- **The Breakthrough:** Because it is purely electrical, jumping from User #1 to User #10,000 is nearly instantaneous.
- **The Result:** Modern DBMS servers exclusively use SSDs because "Random Read" speeds are 100x faster than HDDs.

The Concept of "Pages" (Blocks)

Even with SSDs, going to Disk is millions of times slower than RAM. Therefore, the DBMS **never** fetches a single row. It groups rows into chunks called **Pages** (usually 4KB or 8KB).

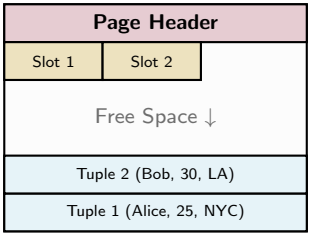


The Book Analogy:
Imagine a massive library. If you want to read a specific sentence, you don't ask the librarian for "Sentence 5". You ask for the **Page**, photocopy the whole Page, and bring it to your desk to read.

Pages (Blocks) on Disk

The Slotted Page Architecture

What does the inside of that 8KB chunk look like? A page isn't just raw data; it is carefully formatted using the **Slotted Page Structure**.

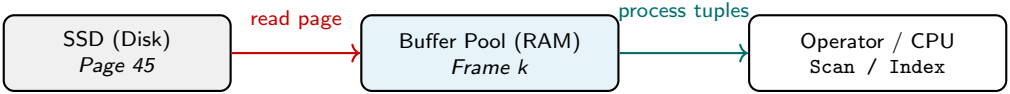


Key Components:

- **Header:** Stores metadata (Page ID, amount of free space).
- **Slot Array:** Grows from top to bottom. It acts as an index, pointing to where each record exactly starts.
- **Tuples (Rows):** The actual data. They grow from the bottom up!

6.5 A Page Read in Motion (Disk → RAM → CPU)

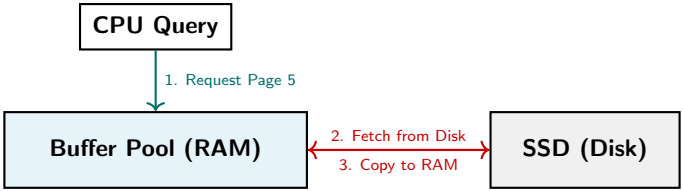
When SQL needs a row, the system moves an entire **page** along this path:



Goal of indexing + optimization: minimize how often the first arrow happens.

The Bouncer between RAM and Disk

The **Buffer Pool** is a dedicated, reserved section of the computer's RAM. The Buffer Pool Manager is the software that controls what Pages get to enter RAM, and what Pages get kicked out.



- If the RAM gets full, the Manager must **Evict** a page to make room (usually tossing out the "Least Recently Used" page).
- If an evicted page was updated (like a user changing their password), the Manager must save the changes to the Disk before destroying it in RAM!

Putting the Hardware Together

You click on your Netflix Profile ("User 999"). What physically happens in the database hardware?

1. The CPU receives the SQL query: `SELECT * FROM Users WHERE ID = 999;`
2. The CPU asks the Buffer Pool Manager: *"Do you have the Page containing User 999 in RAM?"*
3. **Cache Miss:** The Buffer Pool doesn't have it.
4. The Manager goes to the SSD (Disk), finds "Page 45" (which holds Users 900-1000), and copies the entire 8KB page into RAM.
5. The CPU scans the Slotted Page in RAM, finds Tuple #999, extracts your profile, and sends it to your TV!

The Core Metric: Disk I/O Cost

We do not measure database speed by how many rows are read. We measure it by how many Pages had to be fetched from Disk to RAM!

8. The Nightmare: Full Table Scan

Searching without a Map

Imagine a database of 10 Million Airline Flight records, stored across **100,000 Pages** on an SSD.

```
SELECT * FROM Flights WHERE BookingRef = 'ZX99A';
```

How does the DBMS find your flight?

- If the data is just randomly inserted (a Heap File), the DBMS has absolutely no idea which of the 100,000 pages holds your ticket.
- It must load Page 1 to RAM, check it, throw it away... then Page 2... then Page 3.

The Math of a Full Scan:
Fetching 100,000 Pages $\times$ 1ms (Disk read time) = **100 seconds (1.6 minutes)!**
If a user waits 1.6 minutes for their flight booking to appear, they will close the app and go to a competitor.

9. The Solution: Indexing

The Textbook Analogy

How do you find a specific topic in a 1,000-page textbook? Do you read from page 1 until you find it? No, you flip to the **Index** at the back!

What is a Database Index?

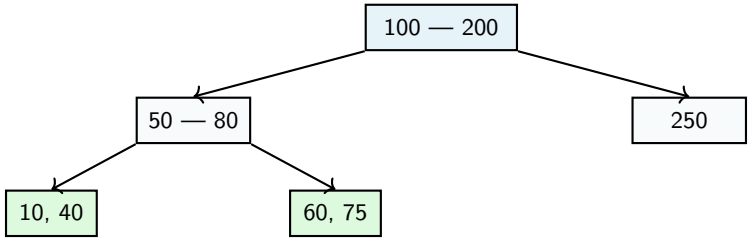
- An Index is a separate, small, highly organized data structure.
- It acts as a map. It maps a Search Key (like your `BookingRef`) directly to the physical **Page ID** where the data lives.

Index File Example:
`Ref 'AA11'` → Page 5
`Ref 'BB22'` → Page 102
`Ref 'ZX99A'` → Page 8450

The Result:
Instead of scanning 100,000 pages, the DBMS looks at the Index, sees you are on Page 8450, and performs **exactly 1 Disk I/O** to fetch your ticket!

How is the Index Structured?

In modern relational databases, the standard index structure is the **B+ Tree**. It is a self-balancing tree that is specifically optimized for Disk-based storage.



- **Massive Fanout:** Because each tree node is the size of an entire 8KB page, a single node can hold hundreds of keys.
- **Shallow Depth:** Even for a table with 1 Billion rows, a B+ Tree is usually only **3 or 4 levels deep**!

11. Visualizing the B+ Tree Search

Finding the Flight Ticket

Let's say we have an index on `FlightID`. We want to find Flight #60.

1. **Read 1 (Root Node):** We load the Root Page to RAM. It says "Values $<$ 100 go Left".
2. **Read 2 (Internal Node):** We load the Left Page to RAM. It says "Values between 50 and 80 go Right".
3. **Read 3 (Leaf Node):** We load the Leaf Page to RAM. This leaf holds the actual pointer to the physical SSD Page where the ticket row exists.

The Performance Miracle

We searched through millions of records by performing exactly **3 Disk I/Os** (Traversing the tree) + **1 Disk I/O** (Fetching the final data page).

Time taken: 4 milliseconds. (Down from 100 seconds!)

12. Query Processing: The Pipeline

How does SQL execute?

Now we know about Storage, Pages, Buffer Pools, and B+ Trees. But when you hit "Enter" on a SQL query, how does the system know which of these tools to use?

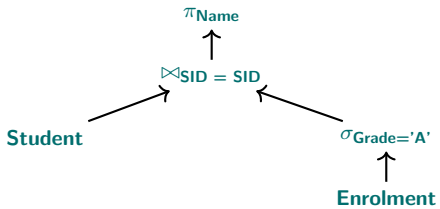
The DBMS Pipeline:

- **Parser:** Checks syntax and translates SQL to Relational Algebra.
- **Optimizer:** The "Brain". Looks at the disks, RAM, and indexes to find the fastest path.
- **Executor:** The "Muscle". Calls the Buffer Pool Manager to fetch the actual pages.

Translating English to Math

SQL is a declarative language (you say *what* you want, not *how* to get it). The Parser converts your SQL text into a **Relational Algebra Tree**.

```
SELECT Name
FROM Student s
JOIN Enrolment e ON s.SID = e.SID
WHERE e.Grade = 'A';
```



The Problem: Relational Algebra tells us the mathematical steps, but it doesn't tell us *physically* how to execute them. Do we scan? Do we index?

14. Step 2: The Optimizer (The Brain)

Does the Optimizer use RAM or Disk?

The Optimizer is a software module running in RAM, but its entire job is to think about the **Disk**.

- There are dozens of mathematically identical ways to execute a query.
- The Optimizer acts like a GPS (Google Maps). It evaluates all possible paths, calculates the **Cost** of each path, and picks the cheapest one.
- **Cost = Estimated number of Disk Page I/Os.**

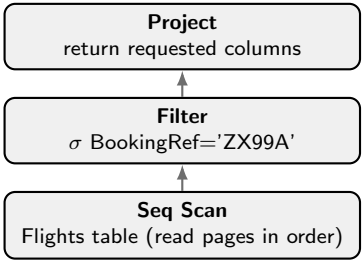
What does the Optimizer decide?

1. **Access Path:** "Should I do a Full Table Scan, or use the B+ Tree Index?"
2. **Join Order:** "Should I join A to B first, or B to C first?"
3. **Join Algorithm:** "Should I use a Nested Loop, or build a Hash Table in RAM?"

14.5 Same SQL, Different Physical Plans

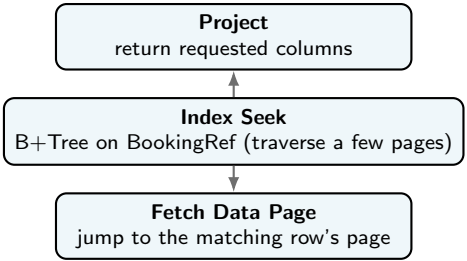
Same query intent: find the row(s) with `BookingRef='ZX99A'`. Two execution plans are **logically equivalent** but have very different **Disk I/O cost**.

Plan A: Full Table Scan



I/O: reads **many** data pages
(e.g., **100,000** pages in the example)

Plan B: Index Lookup



I/O: few index pages + **1** data page
(typically **3–4 + 1** page reads)

15. Optimizer Decision: Scan vs. Index

Is an Index ALWAYS better?

You might think the Optimizer will always choose the B+ Tree Index over a Full Table Scan. **This is false!**

Scenario 1: High Selectivity

Query: `Find the single user with ID = 99.`

- **Selectivity:** We are fetching 1 out of 10 Million records.
- **Decision:** The Optimizer will use the **B+ Tree Index**. We only need to fetch 4 pages!

Scenario 2: Low Selectivity

Query: `Find all users whose Gender = 'Male'.`

- **Selectivity:** We are fetching 50% of the entire database.
- **Decision:** The Optimizer will use a **Full Table Scan**.

Why scan?
If we use an index to fetch 5 million records, the DBMS will have to traverse the tree 5 million times, jumping randomly around the Disk (Random I/O). It is physically faster to just read the entire disk straight through (Sequential I/O)!

16. Optimizer Decision: Filter Pushdown

The Golden Rule of Relational Algebra

The Optimizer will aggressively restructure your mathematical tree to ensure that **Selection (σ)** operations happen as early as possible.

Query: Find all Students in the 'CS' department who got an 'A' in 'DB101'.

Bad Plan (Join First)

- Join all 10,000 Students with all 50,000 Course Enrollments.
- Creates a massive, temporary 500M row table in RAM.
- Filter the massive table for 'CS' and 'DB101'.

Optimized Plan (Filter First)

- Filter Students to just 'CS' (500 rows).
- Filter Enrollments to just 'DB101' 'A's (30 rows).
- Join the 500 rows with the 30 rows. It instantly completes in a tiny fraction of RAM!

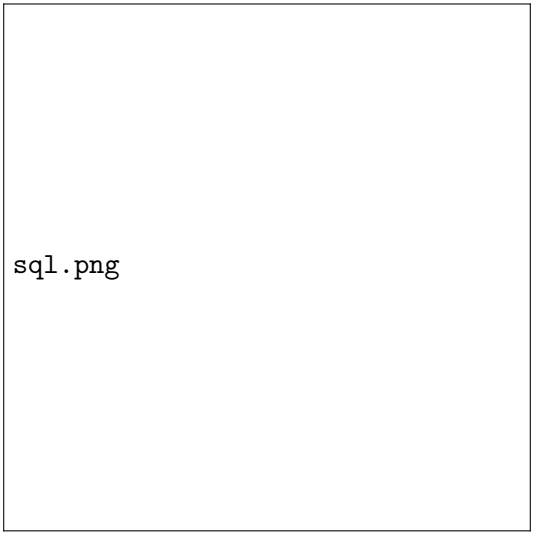
Bringing the Math to Life

Once the Optimizer finalizes the exact "Execution Plan" (e.g., Filter → Hash Join → Project), it hands the instructions to the Execution Engine.

- The Execution Engine does not access the disk directly.
- It pulls data through the **Iterator Model** (also called the Volcano Model).
- Each node in the tree calls a `Next()` function on its child.
- The bottom nodes (Scans/Indexes) ask the **Buffer Pool Manager** for the Pages.
- The Buffer Pool Manager fetches the Pages from the **SSD**.

We have now successfully traced a SQL query from the user's keyboard, through the parsing math, optimized by the brain, requested by the executor, buffered in RAM, and physically read from the flash chips of the SSD!

SQL Query: From Text to Execution



18. Putting it all together (Review)

The Full Stack Vocabulary

If you are asked in an interview how a database works, use these layers:

1. **Disk Space Manager:** Manages the physical files on the HDD/SSD, formatted into 8KB **Pages**.
2. **Buffer Pool Manager:** Moves Pages between the slow Disk and the fast **RAM**. Evicts old pages when full.
3. **Access Methods:** Uses **B+ Trees** or **Table Scans** to find specific rows within the pages.
4. **Query Optimizer:** Translates SQL into math, calculates the Disk I/O costs, and decides whether to use an index or a scan.

19. Summary

Key Takeaways

- **Hardware is King:** All database performance theory revolves around minimizing Disk I/O.
- **SSD over HDD:** Databases prefer SSDs because they lack mechanical seek times, allowing lightning-fast random page reads.
- **Slotted Pages:** The internal architecture of an 8KB block that allows tuples to be accessed by a slot array index.
- **Indexing Saves Apps:** A B+ Tree prevents multi-minute Full Table Scans, dropping lookup times to milliseconds.
- **The Optimizer is Cost-Based:** It does not blindly follow rules; it calculates the hardware cost (pages read) of multiple plans before executing.

20. Next Steps (Road to Q1)

What's Next?

We have covered Design, Storage, and Optimization. But what happens if two people try to buy the same airline ticket at the exact same millisecond?

L18: Transactions & ACID
(Friday, 9 April 2026)

- Concurrency Control (Locking).
- Crash Recovery (Write-Ahead Logs).
- How databases survive power outages.

Revision for ICA 2 & Q1

TA Sri will run a revision session on **Friday, 10 April 2026 (12:00–1:00 PM)** covering:

- Relational Algebra & Relational Calculus
- Normalization (1NF → BCNF/4NF)
- Key earlier topics (Q1 scope): DBMS introduction, database types, ER modeling, relational modeling, and ER→relational schema translation

Reading Assignment

Read **Silberschatz, Ch. 14** (Transactions).